

Essential skills

To be a successful software architect, a software professional should balance technical skills with leadership qualities.

System design: Proficiency in architectural patterns like microservices, monolithic, or event-driven architecture.

Communication: The ability to explain technical trade-offs to non-technical business owners.

Strategic thinking: Comprehend the problem domain to ensure the best software-based solution for the given project.

Leadership and mentoring: Oversee the entire team's work and guide the group through the challenging technical aspects of the project.

Architects typically begin their careers as senior software engineers and are promoted to the position of software architect after five to ten years of experience. Then, rather than how to write a function, they focus on how to solve a problem as a whole.

Software engineers must maintain a balance between technical proficiency and emotional intelligence to become great software architects. An architect is frequently described as a “developer who knows how to talk to management” and “still remembers how to code.”

Here are 10 critical ‘Dos’ and ‘Don’ts’ for the role.

The 5 success drivers

Focus on trade-offs: There are benefits and drawbacks to every architectural decision. The trade-offs of the possibilities should be weighed and the architecture most suited to the project's execution and success should be selected.

Write ADRs (architecture decision records): To make it possible for the customer and other members of the project team to understand the rationale behind the chosen design, it is important to document its background.

Stay ‘hands-on’: To guarantee the implementability of the selected design, the coding process should proceed with prototype implementation, PoCs (proof of concepts), and codebase execution.

Monitor parameters: For the system's long-term health, an eye must be kept on non-functional metrics like security, scalability, and maintainability.

Practice an evolutionary approach: In order to adjust to changes in the future, an evolutionary approach should be taken.

The 5 common pitfalls

Don't be an ‘Ivory Tower’ architect: Discuss with the development team rather than working alone. For a design to be successful, the developers must approve it.

Don't over-engineer: Avoid complex design; if a straightforward monolithic design on a single server can solve the intended business issue, stay away from designs utilising Kubernetes or microservices.

Don't be a bottleneck: Don't design the project with complex and hard to learn technologies. To enable the team to work quickly without you, design the project using straightforward and uncomplicated technology.

Don't ignore future technical risk: The project should never descend into chaos; instead, it should always emphasise the risks with justification and potential remedies.

Don't fall in love with a technology: Avoid being biased towards a technology; even if you don't like it, the best tool is the one that matches the particular project.

Example of an architectural design

ADR 000: Database selection for user profile and activity service

Given below is the architecture decision record (ADR) of a project where the main design task is to select a RDBMS or NoSQL for a user profile and activity tracking system. Since data transfer is much more difficult than rewriting code, selecting a database is one of the most important choices.

- **Status:** Accepted
- **Date:** 2026-01-15
- **Deciders:** Manoj Bhattacharjee (lead architect), Anjan Dutta (data engineer)
- **Consulted:** Backend team, Analytics team

Problem statement

A user appointment system needs to store two types of data:

- **Core profile data:** Highly structured information: user ID, email, subscription plan, password.
- **Activity logs:** Unstructured, high-volume data: clicks, page views, device metadata, and custom event attributes that change frequently.

The task is to decide whether to use a strict relational schema (PostgreSQL) or a flexible document store (MongoDB).

Decision-making factors

- **Data consistency:** Core user data must follow ACID properties: no partial updates to subscription status.
- **Schema flexibility:** Weekly adjustments in tracking points cause activity logs to change. To accommodate all these modifications, the system requires a flexible configuration.
- **Query complexity:** The system requires a process to join user profile data with billing data regularly.
- **Write throughput:** During peak hours, activity logs typically produce a large volume of data (5,000+) per second and grow quite quickly.

The options and the decision

- **Option A: Pure relational (PostgreSQL):** Use standard tables for profiles and JSONB columns for unstructured logs.